Dangal ng Bayan,
bringing pride and honor to the nation.

Pamantasan ng Cabuyao
(University of Cabuyao)

ITEW6
API Integration & Data Handling

Prepared by:

Dr. Luvim M. Eusebio

Intended Learning Outcomes

At the end of this lesson, students should be able to:

1. Explain the concepts of client-side routing and dynamic routes in managing navigation within modern web applications.
2. Differentiate between state and props and describe their roles in controlling data flow and user interaction in component-based frameworks.
3. Demonstrate understanding of global state management concepts and data flow patterns used in large-scale web applications.

What is API Integration?

- An **API (Application Programming Interface)** allows your frontend (UI) to communicate with a backend (server/database).

Example:

- Frontend: React app
- Backend: Node.js API
- Data: User profiles

Flow:

- Frontend → Request → Backend → Response → Frontend UI update

Connecting Frontend to Backend

- Example using fetch:

```
fetch("https://jsonplaceholder.typicode.com/posts")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

Key idea:

- Frontend sends request
- Backend sends data (usually JSON)
- Frontend updates UI

REST vs GraphQL

- **REST API**
- Multiple endpoints
- Fixed data structure

Example:

GET /users

GET /users/1

GraphQL (GraphQL)

- Single endpoint
- Client chooses what data to get

GRAPH QL

```
{
  user(id: 1) {
    name
    email
  }
}
```

Comparison:

Feature	REST	GraphQL
Endpoints	Multiple	Single
Data fetching	Fixed	Flexible
Overfetching	Common	Minimal

HTTP Methods

- These define **what action you want to perform**:

Method	Purpose	Example
GET	Retrieve data	Get users
POST	Create data	Add user
PUT	Update data	Edit user
DELETE	Remove data	Delete user

Example:

```
fetch("/users", {
  method: "POST",
  body: JSON.stringify({ name: "John" }),
  headers: { "Content-Type": "application/json" }
});
```

Fetch vs Axios

- Fetch (built-in)

```
fetch("/api/data")
  .then(res => res.json())
  .then(data => console.log(data));
```

- Axios (Axios)

```
import axios from "axios";

axios.get("/api/data")
  .then(res => console.log(res.data));
```

Comparison:

Feature	Fetch	Axios
Built-in	Yes	No
JSON auto	No	Yes
Simplicity	Moderate	Easier

Handling Async Data

- APIs are **asynchronous** (they take time).
- Using async/await:

```
async function getData() {
  try {
    const response = await fetch("/api/data");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```

Error Handling & Loading States

- **Why important?**

Users need feedback while data is loading or if something fails.

Example:

```
let loading = true;
let error = null;

async function loadData() {
  try {
    const res = await fetch("/api/data");
    const data = await res.json();
    loading = false;
  } catch (err) {
    error = "Failed to load data";
    loading = false;
  }
}
```

ITEW6
Testing in Web Frameworks

Intended Learning Outcomes

At the end of this lesson, students should be able to:

1. Differentiate between development mode and build (production) mode and explain the importance of environment configurations in deployed applications.
2. Describe various hosting platforms and deployment processes used in delivering web applications to production environments.
3. Demonstrate understanding of CI/CD principles and version control workflows in managing, deploying, and maintaining web applications efficiently.

What is Testing (Quality Assurance)?

- **Testing** ensures your application works correctly before users interact with it.

Goal:

- Catch bugs early
- Ensure reliability
- Improve user experience

Types of Testing

- **Unit Testing**
- Tests **small pieces of code** (functions)
  -
```
function add(a, b) {
  return a + b;
}

test("adds numbers correctly", () => {
  expect(add(2, 3)).toBe(5);
});
```
- Fast
- Isolated
- Doesn’t test full UI

Component Testing

- Tests **UI components** (buttons, forms)
- Using React Testing Library:
  - render(<Button />);
    expect(screen.getByText("Click")).toBeInTheDocument();
- Tests UI behavior
- Closer to real usage

End-to-End (E2E) Testing

- Tests **entire user flow**
- Using Cypress:

```
cy.visit("/login");
cy.get("input").type("user");
cy.get("button").click();
```

- Simulates real users
- Slower

Popular Testing Tools

- Jest → unit testing
- React Testing Library → component testing
- Cypress → E2E testing
- Mocha → alternative test runner

Why Testing Matters

Without testing:

- Bugs reach users
- Fixes become expensive
- Features break unexpectedly

With testing:

- Safer code changes
- Better teamwork
- Higher confidence in deployment

THANK YOU FOR LISTENING!!

End of Slide Show